Entrega 6 · comandos y resultados de las pruebas

Todas las ejecuciones se hicieron en equipos Windows con Smart App Control activo, por lo que el código se compiló y probó en contenedores Linux (`mcr.microsoft.com/dotnet/sdk:10.0`). Docker Desktop con los servicios de `docker compose` (PostgreSQL, Redis y RabbitMQ) en marcha. Datos de prueba aislados: Testcontainers crea bases desechables y Cypress/evidencia solo usan pacientes `CYP-E2E-*`.

Pruebas xUnit (unitarias + integración con PostgreSQL, Redis y RabbitMQ reales)

```
scripts/dotnet-linux.sh test MediCore.slnx
```

Proyecto	Resultado
`MediCore.UnitTests`	44 superadas, 0 fallidas, 0 omitidas
`MediCore.Api.Tests`	182 superadas, 0 fallidas, 0 omitidas (incluye `LaboratoryApiTests` y `MessagingTests`)

Cypress (Chrome 155 y Electron 146)

```
scripts/dev-stack-linux.sh up        # API y web en contenedores Linux, sobre las dependencias de compose
cd tests/cypress && CYPRESS_MEDICORE_PASSWORD=<Seed:DevPassword> npx cypress run --browser chrome
cd tests/cypress && CYPRESS_MEDICORE_PASSWORD=<Seed:DevPassword> npx cypress run --browser electron
scripts/dev-stack-linux.sh down
```

Spec	Chrome	Electron
`laboratorio-resultados.cy.js` (nuevo)	8/8	8/8
`medico-expediente.cy.js`	10/10	10/10
`recepcion-reserva.cy.js` (regresión)	8/8	8/8

Evidencia con el broker real

```
MEDICORE_PASSWORD=<Seed:DevPassword> python scripts/evidencia_laboratorio.py
```

Informe: `documentacion/evidencias/entrega-6-laboratorio.md` (broker disponible; broker detenido y recuperado; reentrega sin duplicados; 13/13 comprobaciones de autorización y validación).

Pruebas de integración de mensajería (`MessagingTests`, RabbitMQ en Testcontainers)

1. Registro → publicación → consumo → notificación interna del paciente.

2. RabbitMQ detenido: el resultado se guarda y queda "Pendiente"; al volver el broker el evento se recupera y se notifica.
3. Mensaje reentregado con el mismo `MessageId` y mensaje nuevo para el mismo resultado: una sola notificación.
4. Evento del outbox publicado dos veces: una sola notificación.
5. El consumidor falla dos veces: reintenta con espera y termina procesando (3 entregas).
6. El consumidor falla siempre: el mensaje va a la DLQ con el tipo de error (`InvalidOperationException`) y sin notificación; tras corregir la causa, "reintentar" encola un evento nuevo y la notificación se crea una sola vez.
7. Mensaje ilegible o de un resultado inexistente: directo a la DLQ, sin reintentos.
8. Publicador y consumidor activos con broker y apagados sin él.